

ChessRabbit

A chess database and analysis platform — game database, server-side Stockfish analysis, opening explorer, and PGN management, delivered as a web app with free and Pro tiers.

The screenshot displays the ChessRabbit web application interface. At the top, a navigation bar includes links for Database, Analyse, Explorer, Train, Prep, and Insights, along with a PRO plan button. The main workspace is divided into several sections:

- Chessboard:** A standard 8x8 chessboard with pieces in their starting positions. A vertical evaluation bar on the left shows a score of +0.34. Below the board, it indicates "1. e4 — White to move, depth 24".
- Engine · Stockfish 16 (server-side):** Displays engine statistics (depth 24, nps 18.4M, 2 threads, MultiPV 3) and three recommended move lines with their respective scores: +0.34 for 1... c5, +0.29 for 1... e5, and +0.11 for 1... e6.
- Move tree · variations, annotations:** Shows a sequence of moves: 1. e4, 2. Nf3, 3. Bb5, 4. Ba4, 5. O-O, 6. Re1. It also lists variations like e5, Nc6, a6, Nf6, Be7, and b5?, with annotations such as "engine best" and "blunder -1.8 → puzzle card created".
- Opening explorer · masters | my games:** A table showing the frequency of various openings in master games and the user's own games.

Opening	Masters	My Games
c5	Sicilian	41%
e5	Open Game	33%
e6	French	14%
c6	Caro-Kann	8%

The analysis workspace: eval bar, board, engine lines, annotated move tree with variations, and the opening explorer — all fed by one server-side engine.

What it is

ChessRabbit is a self-hostable chess study platform. You import your games, the server analyses them with Stockfish, and everything the engine finds is turned into something you can act on: annotated games, a personal opening tree next to master statistics, spaced-repetition cards built from your own blunders, and opponent preparation. The engine never runs in the browser — it runs as an unmodified binary in its own process on

the server, which is both a licensing requirement and the reason analysis quality does not depend on the user's laptop.

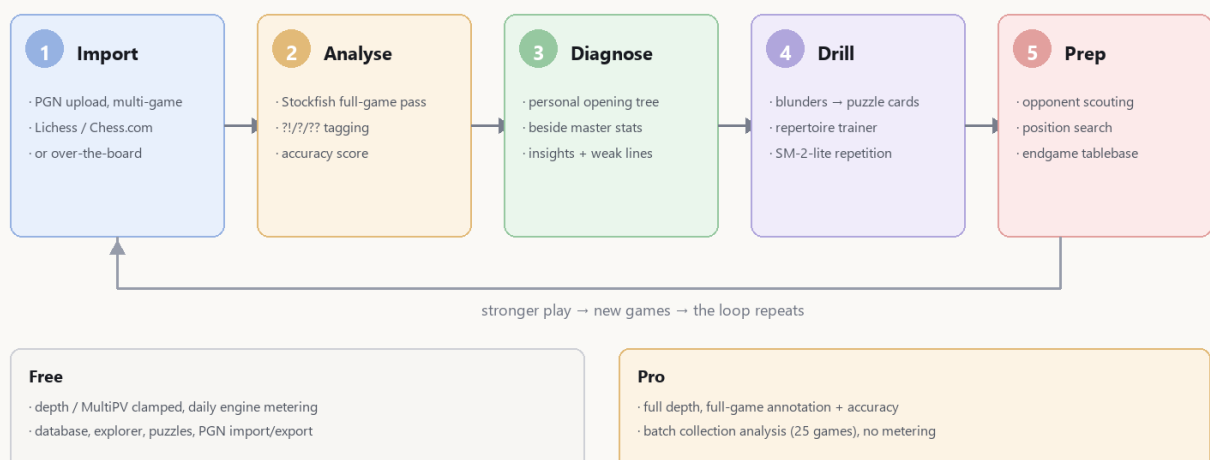
At a glance

Area	What ships today
Accounts	Register, login, rotating refresh tokens, logout, email verification, enumeration-safe password reset with single-use tokens, Redis fixed-window rate limiting on auth endpoints.
Database	Multi-game PGN import, listing, detail, delete, annotations, collections, per-game PGN export that round-trips, search by player / result / ECO.
Analysis	Queued position analysis through Stockfish with a Redis eval cache (~10 ms on a hit), a live WebSocket that streams progressive depth, full-game annotation with ?!/?? tagging and accuracy scores, batch collection analysis.
Openings	Opening explorer and position search over a reference database, plus a “My games” scope that shows your own move statistics beside the master reference.
Training	Blunder puzzles generated from engine-tagged mistakes, a repertoire trainer built from PGNs with variations (transpositions dedupe on Zobrist key), SM-2-lite spaced repetition, answers judged server-side.
Endgames	Tablebase endpoint: rule-based draws work out of the box; point SYZGY_PATH at Syzygy 3-4-5 files for full WDL/DTZ probing.
Business	Stripe checkout and customer portal, a webhook handler that mirrors subscription state and flips free/pro, plus an admin surface: stats, user roster with usage, suspend/unsuspend.

How you actually use it

The product is organised around a loop rather than a set of unrelated tools. Every stage consumes what the previous stage produced, so the material you train on is always drawn from your own play instead of a generic course.

The improvement loop: your own games become your training material



Import → analyse → diagnose → drill → prepare, and the tiers that gate engine depth and batch work.

Import

Games come in as PGN — a single game or a whole archive. Insights work for any player: a Lichess handle, a Chess.com handle, or games played over the board and typed in by hand.

Analyse

A full-game pass walks every position through the engine, tags inaccuracies, mistakes and blunders, and produces an accuracy score for each side. Free accounts get clamped depth and MultiPV with daily metering; Pro removes the clamp and unlocks annotation and batch analysis.

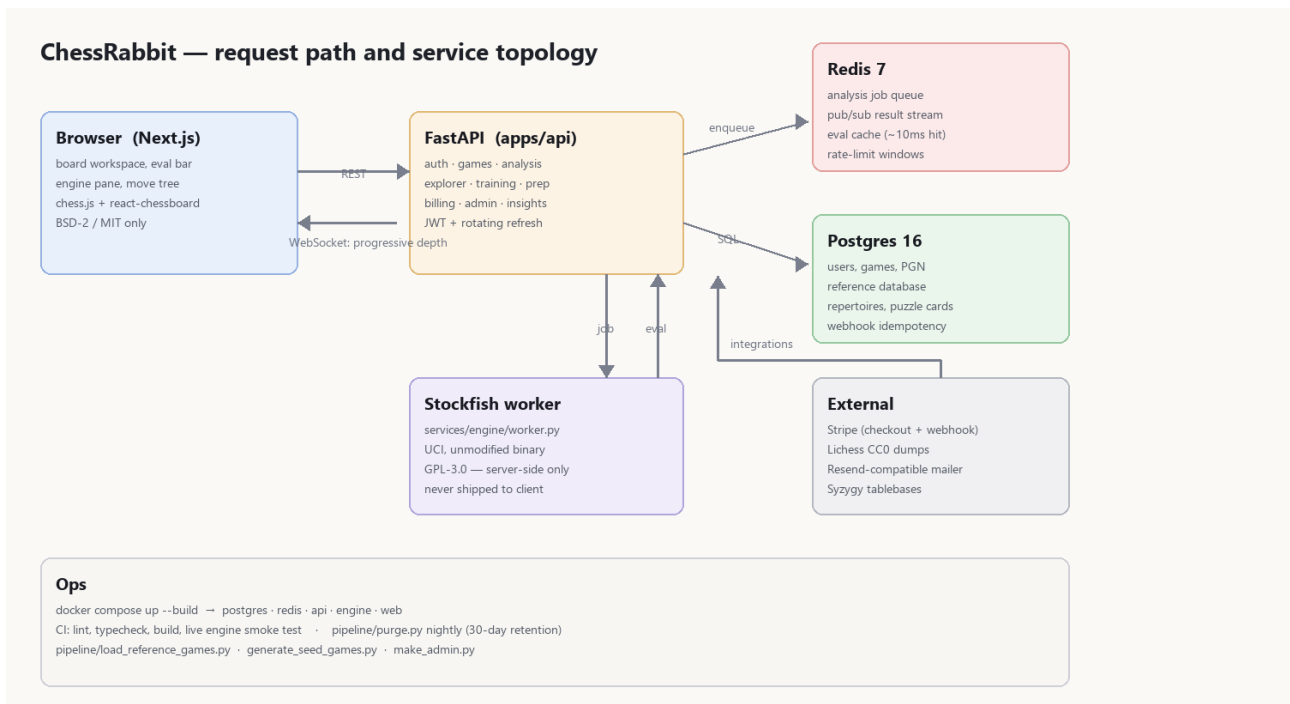
Diagnose

The personal opening tree puts your own results next to the master reference for the same position, which is where most of the useful information lives: not “what do grandmasters play here” but “where do my results diverge from theirs.”

Drill

Every ? or ?? the engine tags becomes a spaced-repetition card in an automatic “My Blunders” repertoire, with the engine's best move stored as the expected answer and judged server-side. Repertoires built from your own PGNs work the same way: opponent branches expand, transpositions collapse onto the same node.

How it is built



Five services in the development compose file, and the boundary that keeps the GPL engine on the server.

The analysis path

- The browser asks the API for an evaluation. If the position is already in the Redis eval cache, the answer comes straight back — cache hits are answered in roughly ten milliseconds.

- On a miss the API enqueues a job in Redis. The Stockfish worker pops it, talks UCI to an unmodified engine binary, and publishes results to a pub/sub channel as depth increases.
- The API relays that stream over a WebSocket, so the eval bar and engine lines deepen in front of the user instead of appearing all at once.
- Finished evaluations are written to the cache and, where they belong to a stored game, to Postgres.

Stack

Layer	Technology	Notes
Web	Next.js, chess.js, react-chessboard	Shipped as a static bundle; board sizes to min(92vw, 480px) so it fits phones
API	FastAPI (Python)	auth, games, analysis, explorer, training, prep, insights, billing, admin, WS
Engine	Stockfish via UCI	Separate worker process, Redis queue in and pub/sub out
Data	Postgres 16, Redis 7	SQL migrations in db/migrations, applied on first boot
Email	Provider-agnostic mailer	Console backend in dev, Resend-compatible HTTP backend when EMAIL_API_KEY is set
Payments	Stripe	Checkout, customer portal, signed webhooks

Repository layout

```

BLUEPRINT.md      product + technical blueprint (read this first)
db/migrations/    SQL schema
apps/api/         FastAPI backend (auth, games, analysis, explorer, WS)
apps/web/         Next.js frontend (board workspace, auth pages)
services/engine/  Stockfish worker: Redis queue → UCI → pub/sub + cache
pipeline/         reference-database loader (Lichess CC0 dumps), seeding, maintenance
.github/workflows/ CI: lint, live engine smoke test, typecheck, build

```

Running it

The whole stack comes up with one command. Postgres migrations are applied automatically on first boot.

```

cp .env.example .env
# set JWT_SECRET at minimum: openssl rand -hex 32
docker compose up --build

```

Service	URL
Web	http://localhost:3000
API	http://localhost:8000
API docs	http://localhost:8000/docs

Filling the explorer

In production the reference database is loaded from the Lichess open database (CC0). For development there is a faster path: Stockfish plays itself out of standard opening theory and generates realistic seed games in about two minutes, instead of downloading gigabytes.

```
python pipeline/generate_seed_games.py --games 60 --out seed.pgn
python pipeline/load_reference_games.py seed.pgn
```

Licensing boundary

This is the one design rule that cannot be relaxed, because it is what keeps the application code proprietary.

- **Stockfish is GPL-3.0** and runs **server-side only**, as an unmodified binary in its own process. It is never bundled into the frontend, never compiled to WASM, never shipped in any client artifact.
- python-chess is GPL-3.0 and is likewise server-side only.
- chess.js (BSD-2) and react-chessboard (MIT) are the only chess code that reaches a browser.
- The /open-source endpoint serves the attribution page required at launch.
- “ChessBase” is someone else's trademark and is not used in product names or marketing copy.

Before it takes real traffic

- Rotate every credential that has ever been in a shell, a file, or the repository. The development placeholders are published in the repo, and git keeps whatever was committed even after it is edited out — rewriting a file does not rewrite history.
- Never run the demo-user seeder against a real database: it creates an admin account whose password is printed in the repository. It refuses to run unless ENVIRONMENT=development is set explicitly.
- Billing correctness is enforced by design and covered by tests: the plan flip happens only in the webhook handler, signatures are verified with constant-time HMAC comparison and a five-minute tolerance, and every event id is recorded so Stripe's retries cannot double-process.
- Run the nightly purge job: it hard-deletes accounts past the 30-day retention window and sweeps expired tokens.

Where it stands

Everything above is implemented and covered by tests that ran against live services. What remains is operational rather than code: real Stripe keys, real Lichess data, Syzygy tablebase files, an email provider key, and a server to deploy on.